MiniEVM Optimistic Rollup 구현 분석

1. Optimistic Rollup 개요

MiniEVM은 OPinit 스택을 기반으로 한 Optimistic Rollup Consumer Chain으로 구현되어 있습니다. 이는 L1 체인의 보안을 상속받으면서도 높은 처리량과 낮은 수수료를 제공합니다.

1.1 Optimistic Rollup 원리

- **낙관적 실행**: 모든 트랜잭션이 유효하다고 가정하고 실행
- **사기 증명**: 잘못된 트랜잭션 발견 시 증명을 통해 롤백
- **챌린지 기간**: 일정 기간 동안 누구나 잘못된 상태에 대해 이의 제기 가능
- **L1 보안 상속**: L1 체인의 보안성을 그대로 활용

1.2 MiniEVM의 Optimistic Rollup 특징

- **OPinit 스택 활용**: Initia Labs에서 개발한 Optimistic Rollup 프레임워크
- **검증자 없는 운영**: 별도의 검증자 그룹 불필요
- **즉시 확정성**: CometBFT를 통한 빠른 트랜잭션 확정
- **EVM 호환성**: Ethereum 트랜잭션을 Optimistic Rollup에서 실행

2. OPChild 모듈 구조

2.1 OPChild Keeper

```
type Keeper struct {
```

```
// 기본 필드들
cdc codec.BinaryCodec
storeService corestoretypes.KVStoreService
// 다른 모듈 Keeper들
accountKeeper types.AccountKeeper
bankKeeper types.BankKeeper
oracleKeeper types.OracleKeeper
// L1 브리지 관련
anteHandler sdk.AnteHandler
txDecoder sdk.TxDecoder
msgRouter baseapp.MessageRouter
// 권한 관리
authority  string
// 주소 코덱들
ac address.Codec
vc address.Codec
cc address.Codec
logger log.Logger
}
```

2.2 OPChild 모듈 초기화

```
appKeepers.OPChildKeeper = opchildkeeper.NewKeeper(
```

```
appCodec,
runtime.NewKVStoreService(appKeepers.keys[opchildtypes.StoreKey]),
appKeepers.AccountKeeper,
appKeepers.BankKeeper,
appKeepers.OracleKeeper,
ante.CreateAnteHandlerForOPinit(appKeepers.AccountKeeper, txConfig.SignModeHandl
txConfig.TxDecoder(),
bApp.MsgServiceRouter(),
authorityAddr,
ac,
vc,
cc,
logger,
)
```

3. L1-L2 브리지 시스템

3.1 브리지 정보 구조

```
type BridgeInfo struct {
// L1 체인 정보
L1ChainId  string
L1ClientId  string
```

```
L1ChannelId  string

// 브리지 설정

BridgeId  uint64

BridgeConfig BridgeConfig

// 시퀀스 번호

NextL1Sequence  uint64

NextL2Sequence  uint64

}
```

3.2 토큰 출금 프로세스

```
// ERC20 래퍼에서 L1으로 토큰 출금 요청

message = string(

abi.encodePacked(

'{"@type": "/opinit.opchild.v1.MsgInitiateTokenWithdrawal"',

',"sender": "', sender, '"',

',"to": "', to, '"',

',"amount": { "denom": "',

COSMOS_CONTRACT.to_denom(token),

'", "amount": "', amount.toString(), '" }',

'}'

)
```

```
);
```

3.3 L1에서 L2로 토큰 입금

L1에서 토큰을 예치하면 L2에서 해당 토큰이 발행됩니다:

1. **L1 예치**: 사용자가 L1에서 토큰을 브리지 컨트랙트에 예치
2. **메시지 전송**: IBC를 통해 L2로 입금 메시지 전송
3. **L2 발행**: L2에서 해당 토큰을 사용자에게 발행
4. **상태 업데이트**: 브리지 상태 및 시퀀스 번호 업데이트

4. 검증자 관리 시스템

4.1 검증자 구조

```go
type Validator struct {
OperatorAddress  string
ConsPubkey *types.Any
Jailed  bool
Status BondStatus
Tokens  string
DelegatorShares  string
Description Description
UnbondingHeight  int64
UnbondingTime time.Time
Commission Commission
```

```
MinSelfDelegation  string

Moniker  string

}
```

4.2 검증자 세트 업데이트

```
// IBC 클라이언트 업데이트 후 호출되는 후크

func (k Keeper) UpdateHostValidatorSet(ctx sdk.Context, clientID  string, consen

// L1에서 검증자 세트 변경 사항을 가져와서 L2에 적용

validators, err := k.GetValidatorsFromL1(ctx, clientID, consensusHeight)

if  err != nil {

return  err

}

// L2 검증자 세트 업데이트

return  k.UpdateValidatorSet(ctx, validators)

}
```

4.3 검증자 없는 운영

MiniEVM은 전통적인 검증자 그룹 없이 운영됩니다:

```
// 검증자 대신 OPChild 모듈이 스테이킹 인터페이스 구현

func (app *MinitiaApp) GetStakingKeeper() ibctestingtypes.StakingKeeper {

return  app.OPChildKeeper
```

```
}
```

5. 가스 가격 및 수수료 관리

5.1 최소 가스 가격 설정

```
type Params struct {
// 최소 가스 가격
MinGasPrices sdk.DecCoins
// 브리지 실행자 목록
BridgeExecutors []string
// 관리자 주소
Admin  string
// 후크 실행 최대 가스
HookMaxGas  uint64
}
```

5.2 가스 가격 Keeper

```
type GasPriceKeeper struct {
opck opchildKeeperForGasPriceKeeper
}
```

```go
func (gk GasPriceKeeper) MinGasPrices(ctx context.Context) (sdk.DecCoins, error)
return  gk.opck.MinGasPrices(ctx)
}
```

5.3 수수료 검증

```go
// OPChild 모듈의 수수료 검증기
txFeeChecker := opchildante.NewMempoolFeeChecker(app.OPChildKeeper).CheckTxFeeWi

// Free Lane을 위한 커스텀 수수료 검증
freeLaneFeeChecker := func(ctx sdk.Context, tx sdk.Tx) (sdk.Coins, int64, error)
if !freeLane.Match(ctx, tx) {
return  txFeeChecker(ctx, tx)
}
// Free Lane의 경우 수수료 검증 없이 통과
feeTx, ok := tx.(sdk.FeeTx)
if !ok {
return  nil, 0, errorsmod.Wrap(sdkerrors.ErrTxDecode, "Tx must be a FeeTx")
}
return  feeTx.GetFee(), 1  /* FIFO */, nil
}
```

6. Lane 기반 트랜잭션 처리

6.1 Lane 구조

MiniEVM은 여러 Lane을 통해 트랜잭션을 분류하고 처리합니다:

```
// System Lane: 시스템 트랜잭션 (최고 우선순위)
systemLane := initialanes.NewSystemLane(blockbase.LaneConfig{
Logger: app.Logger(),
TxEncoder: app.txConfig.TxEncoder(),
TxDecoder: app.txConfig.TxDecoder(),
MaxBlockSpace: math.LegacyMustNewDecFromStr("0.1"),
MaxTxs: 1,
SignerExtractor: signerExtractor,
}, opchildlanes.SystemLaneMatchHandler())

// MEV Lane: MEV 경매 트랜잭션
mevLane := initialanes.NewMEVLane(blockbase.LaneConfig{
Logger: app.Logger(),
TxEncoder: app.txConfig.TxEncoder(),
TxDecoder: app.txConfig.TxDecoder(),
MaxBlockSpace: math.LegacyMustNewDecFromStr("0.1"),
MaxTxs: 100,
SignerExtractor: signerExtractor,
}, factory, factory.MatchHandler())
```

```
// Free Lane: 수수료 없는 트랜잭션

freeLane := initialanes.NewFreeLane(blockbase.LaneConfig{

Logger: app.Logger(),

TxEncoder: app.txConfig.TxEncoder(),

TxDecoder: app.txConfig.TxDecoder(),

MaxBlockSpace: math.LegacyMustNewDecFromStr("0.1"),

MaxTxs: 100,

SignerExtractor: signerExtractor,

}, opchildlanes.NewFreeLaneMatchHandler(app.ac, app.OPChildKeeper).MatchHandler(



// Default Lane: 일반 트랜잭션

defaultLane := initialanes.NewDefaultLane(blockbase.LaneConfig{

Logger: app.Logger(),

TxEncoder: app.txConfig.TxEncoder(),

TxDecoder: app.txConfig.TxDecoder(),

MaxBlockSpace: math.LegacyMustNewDecFromStr("0.7"),

MaxTxs: mempoolMaxTxs,

SignerExtractor: signerExtractor,

})
```

6.2 Free Lane 매칭

```
// Free Lane 매칭 핸들러
```

```
freeLaneHandler := opchildlanes.NewFreeLaneMatchHandler(app.ac, app.OPChildKeepe
```

```
// 특정 조건의 트랜잭션을 Free Lane으로 라우팅
func (h *FreeLaneMatchHandler) MatchHandler() func(sdk.Context, sdk.Tx) bool {
return  func(ctx sdk.Context, tx sdk.Tx) bool {
// 브리지 관련 트랜잭션이나 시스템 트랜잭션은 Free Lane 사용
return  h.isBridgeTransaction(ctx, tx) || h.isSystemTransaction(ctx, tx)
}
}
```

7. Ante Handler 확장

7.1 OPChild Ante Decorator

```
// 중복 브리지 트랜잭션 방지
opchildante.NewRedundantBridgeDecorator(options.OPChildKeeper)
```

7.2 브리지 트랜잭션 검증

```
type RedundantBridgeDecorator struct {
opchildKeeper OpChildKeeper
}
```

```
func (rbd RedundantBridgeDecorator) AnteHandle(ctx sdk.Context, tx sdk.Tx, simul
```

```
// 브리지 트랜잭션의 중복 실행 방지
for _, msg := range tx.GetMsgs() {
if bridgeMsg, ok := msg.(*opchildtypes.MsgFinalizeTokenDeposit); ok {
// 이미 처리된 브리지 트랜잭션인지 확인
if rbd.opchildKeeper.IsProcessedBridgeTx(ctx, bridgeMsg.Sequence) {
return ctx, errors.New("redundant bridge transaction")
}
}
}
return next(ctx, tx, simulate)
}
```

8. 상태 증명 및 검증

8.1 상태 루트 제출

L2의 상태 루트는 주기적으로 L1에 제출됩니다:

```
type StateRoot struct {
Height uint64
StateRoot []byte
Timestamp time.Time
Proposer string
```

```
}
```

8.2 사기 증명 메커니즘

```
type FraudProof struct {
// 잘못된 상태 전환 정보
InvalidStateTransition StateTransition
// 증명 데이터
Proof []byte
// 챌린지 기간
ChallengeWindow time.Duration
// 증명 제출자
Challenger  string
}
```

9. IBC 통합

9.1 L1과의 IBC 연결

```
// IBC Keeper 설정 시 OPChild를 스테이킹 모듈로 사용
appKeepers.IBCKeeper = ibckeeper.NewKeeper(
appCodec,
appKeepers.keys[ibcexported.StoreKey],
nil, // migration 불필요
```

```
appKeepers.OPChildKeeper, // 스테이킹 모듈 대신 사용
appKeepers.UpgradeKeeper,
appKeepers.ScopedIBCKeeper,
authorityAddr,
)
```

9.2 검증자 세트 업데이트 후크

```
// IBC 클라이언트 업데이트 후 검증자 세트 동기화
appKeepers.IBCKeeper.ClientKeeper.SetPostUpdateHandler(
appKeepers.OPChildKeeper.UpdateHostValidatorSet,
)
```

10. 토큰 생성 및 관리

10.1 토큰 생성 함수 등록

```
// ERC20 Keeper의 토큰 생성 함수를 OPChild에 등록
appKeepers.OPChildKeeper.WithTokenCreationFn(erc20Keeper.TokenCreationFn)
```

10.2 자동 토큰 생성

L1에서 새로운 토큰이 브리지될 때 자동으로 L2에서 해당 ERC20 토큰이 생성됩니다:

```go
func (k *ERC20Keeper) TokenCreationFn(ctx context.Context, denom  string, metada

// 새로운 ERC20 토큰 자동 생성

contractAddr, err := k.CreateERC20(ctx, denom)

if  err != nil {

return  err

}

// 메타데이터 설정

return  k.SetERC20Metadata(ctx, contractAddr, metadata)

}
```

11. 보안 및 검증

11.1 브리지 실행자 권한

```go
type Params struct {

// 브리지 트랜잭션을 실행할 수 있는 주소 목록

BridgeExecutors []string

// 관리자 주소 (파라미터 변경 권한)

Admin  string

}
```

11.2 권한 검증

```go
func (k Keeper) ValidateBridgeExecutor(ctx context.Context, executor  string) er

params, err := k.GetParams(ctx)

if  err != nil {

return  err

}

for  _, validExecutor := range  params.BridgeExecutors {

if  executor == validExecutor {

return  nil

}

}

return  errors.New("unauthorized bridge executor")

}
```

12. 모니터링 및 메트릭

12.1 브리지 상태 추적

```go
// 다음 L1/L2 시퀀스 번호 조회

func (k Keeper) GetNextL1Sequence(ctx context.Context) (uint64, error)

func (k Keeper) GetNextL2Sequence(ctx context.Context) (uint64, error)


// 브리지 정보 조회
```

```go
func (k Keeper) GetBridgeInfo(ctx context.Context) (*BridgeInfo, error)
```

12.2 검증자 상태 모니터링

```go
// 검증자 정보 조회
func (k Keeper) GetValidator(ctx context.Context, operatorAddr  string) (*Valida

// 모든 검증자 조회
func (k Keeper) GetAllValidators(ctx context.Context) ([]Validator, error)
```

13. 결론

MiniEVM의 Optimistic Rollup 구현은 다음과 같은 특징을 가집니다:

13.1 핵심 장점

- **검증자 없는 운영**: 복잡한 검증자 관리 불필요
- **L1 보안 상속**: L1 체인의 보안성을 그대로 활용
- **즉시 확정성**: CometBFT를 통한 빠른 트랜잭션 처리
- **EVM 호환성**: 기존 Ethereum 도구와 완전 호환

13.2 기술적 혁신

- **Lane 기반 처리**: 트랜잭션 타입별 최적화된 처리
- **자동 토큰 브리징**: L1-L2 간 원활한 토큰 이동
- **IBC 통합**: Cosmos 생태계와의 완전한 연결

- **유연한 수수료**: Free Lane을 통한 수수료 최적화

이러한 구현을 통해 MiniEVM은 Optimistic Rollup의 모든 장점을 활용하면서도 개발자와 사용자에게 최고의 경험을 제공합니다.